9. a. Multi – level Linked List

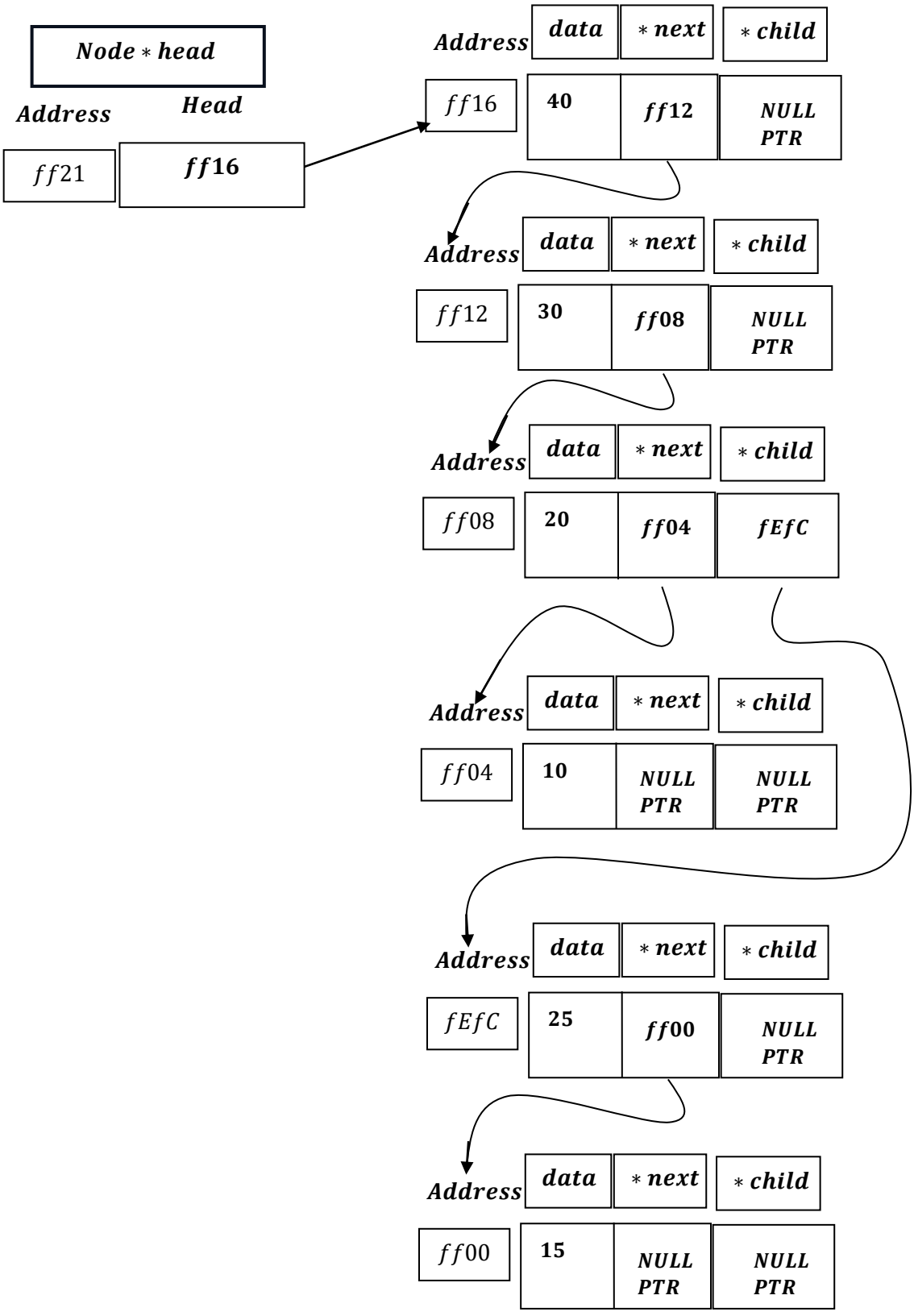
–Print List(Main List and Child)

–Program Analysis

Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}
….
       case 9:
            cout << "Current List Structure: " << endl;
            printList(head);
            break;
```

Program Analysis



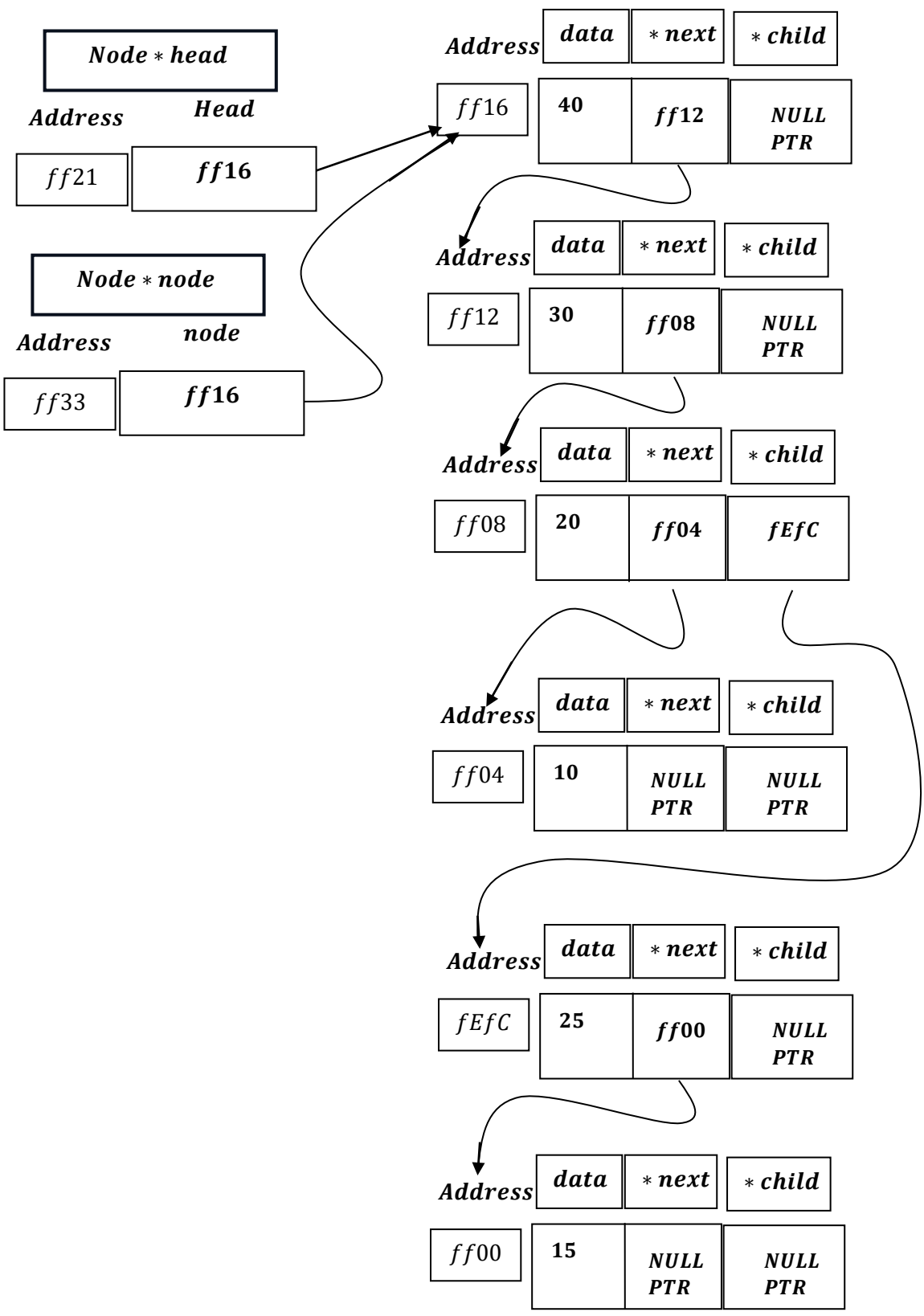
```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
        {
            cout << "    ";
        }

        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }

        node = node->next;
    }
}
```

$$void\ \ printlist(Node * node, int\ level = 0)$$

Function Call:

```
            printList(head);
```

$$Hence\ Node * node = head = ff16.$$


Node * head
Address
Head
ff21
ff16
Node * node
Address
node
ff33
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

$$while\ node{:}\ ff16\ \neq Nullptr\ , which\ is\ true{:}$$

$$for(\ int\ i = 0\ ; i{:}\ 0 < level{:}\ 0\ [false\ condition]; i++) \rightarrow$$

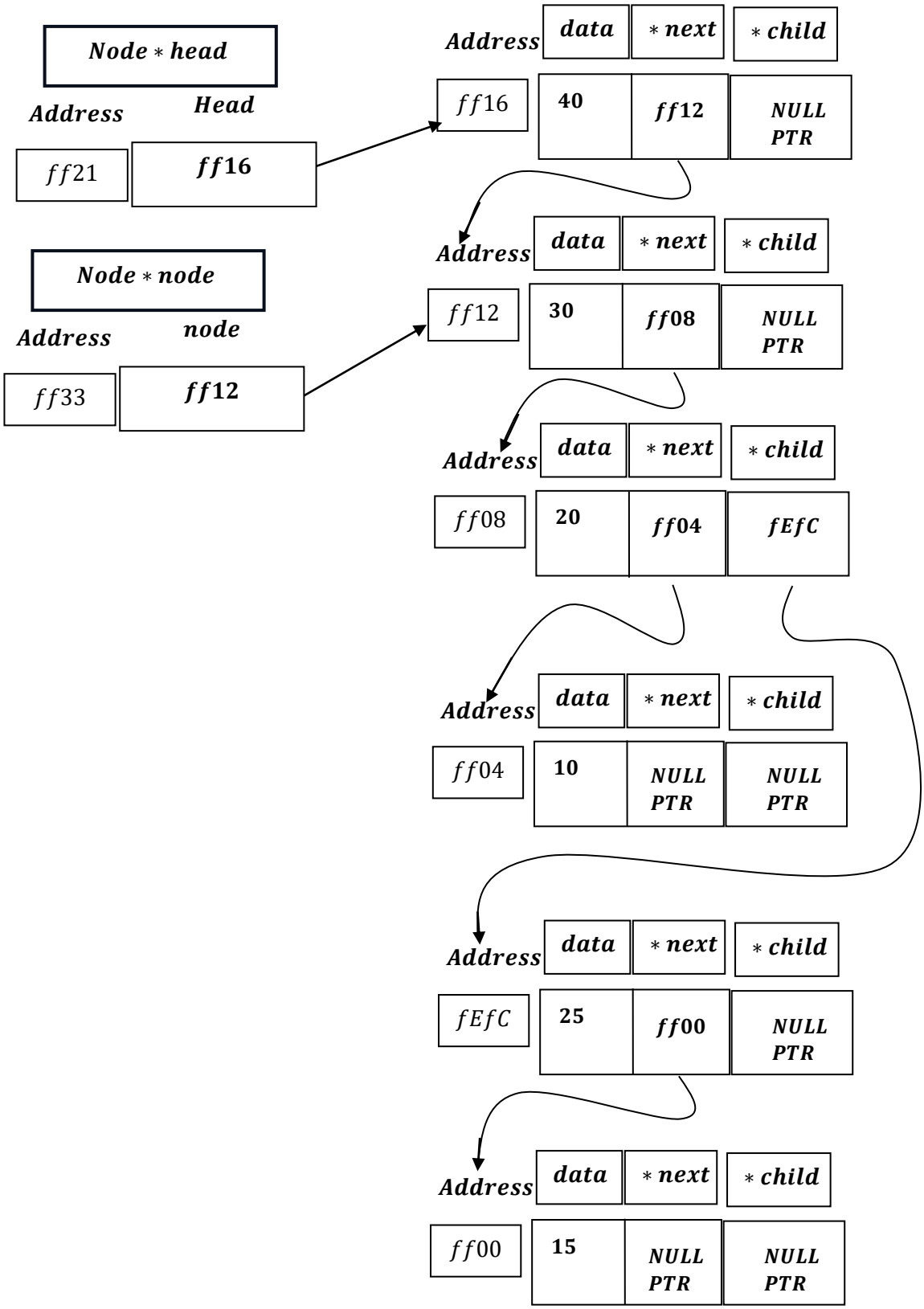
$$(condition\ is\ false, hence\ loop\ doesn't\ run)$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node{:}\ ff16 \rightarrow data{:}\ 40 \ll endl;$$

$$if(node{:}\ ff16 \rightarrow child{:}\ Nullptr \neq NULLPTR) which\ is\ false\ .$$

$$Node = Node \rightarrow next{:}\ ff12.$$


Node * head
Address
Head
ff21
ff16
Node * node
Address
node
ff33
ff12
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

$$while\ node{:}\ ff12\ \neq Nullptr\ , which\ is\ true{:}$$

$$for(\ int\ i = 0\ ; i{:}\ 0 < level{:}\ 0\ [false\ condition]; i++) \rightarrow$$

$$(condition\ is\ false, hence\ loop\ doesn't\ run)$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node{:}\ ff12 \rightarrow data{:}\ 30 \ll endl;$$

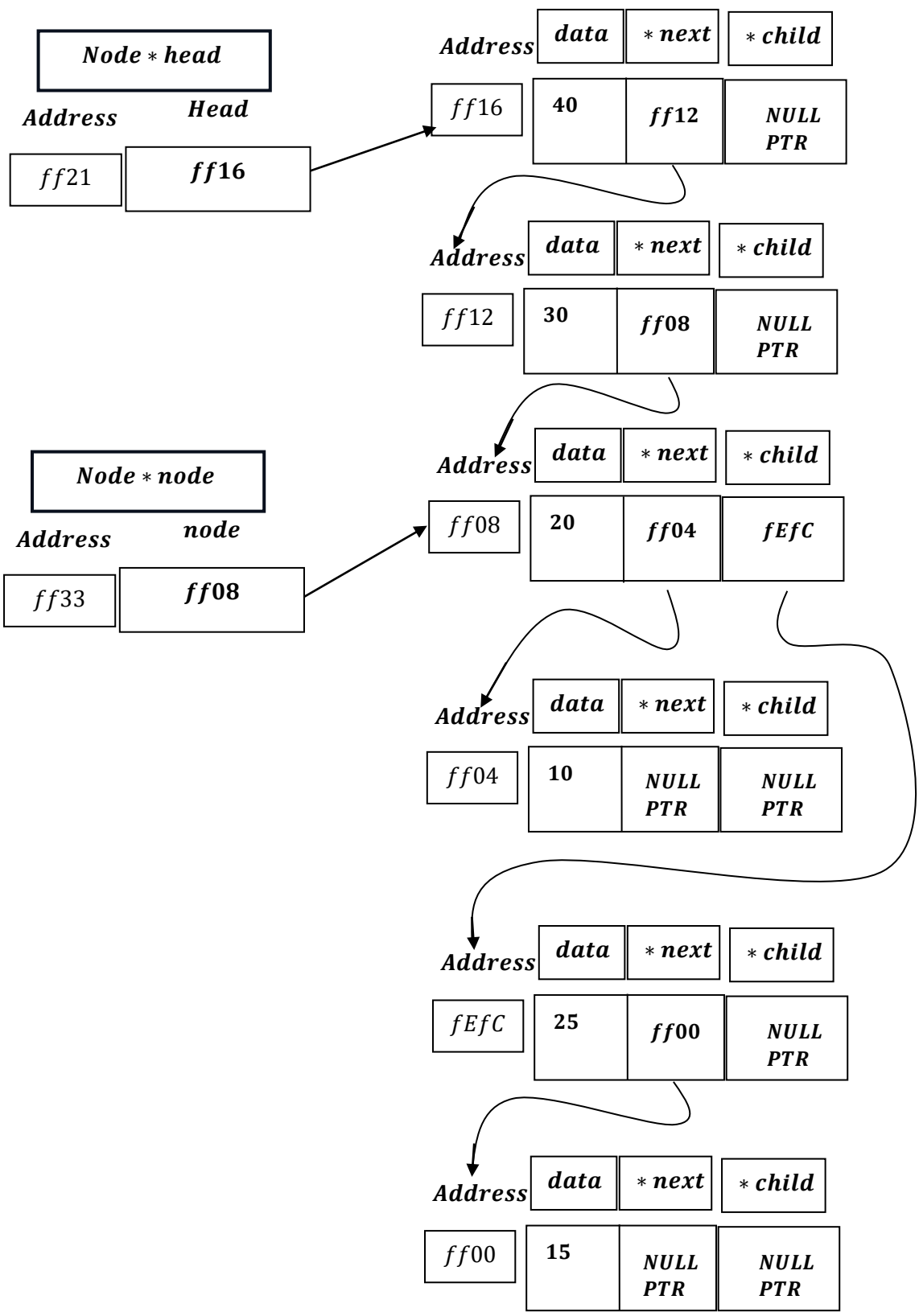
$$if(node{:}\ ff12 \rightarrow child{:}\ Nullptr \neq NULLPTR) which\ is\ false\ .$$

$$Node = Node \rightarrow next{:}\ ff08.$$

$$Upto\ this\ it\ is\ printed{:}$$

$$|--40$$

$$|--30$$


Node * head
Address
Head
ff21
ff16
Node * node
Address
node
ff33
ff08
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

$$while\ node{:}\ ff08\ \neq Nullptr\ , which\ is\ true{:}$$

$$for(\ int\ i = 0\ ; i{:}\ 0 < level{:}\ 0\ [false\ condition]; i++) \rightarrow$$

$$(condition\ is\ false, hence\ loop\ doesn't\ run)$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node{:}\ ff08 \rightarrow data{:}\ 20 \ll endl;$$

$$if(node{:}\ ff08 \rightarrow child{:}\ fEfC \neq NULLPTR) which\ is\ True:$$

```
                printList(node->child, level + 1);
```

$$printList(node \rightarrow child{:}\ fEfC, Level{:}\ 0 + 1 = 1)$$

$$As\ soon\ as\ it\ is\ called\ recursively{:}$$

$printList(node \rightarrow child{:}\ fEfC, Level{:}\ 0 + 1 = 1) \rightarrow Stack\ Frame$

$RecursiveCall\ Stack(PUSH)$

$$Upto\ this\ it\ is\ printed{:}$$

$$|--40$$

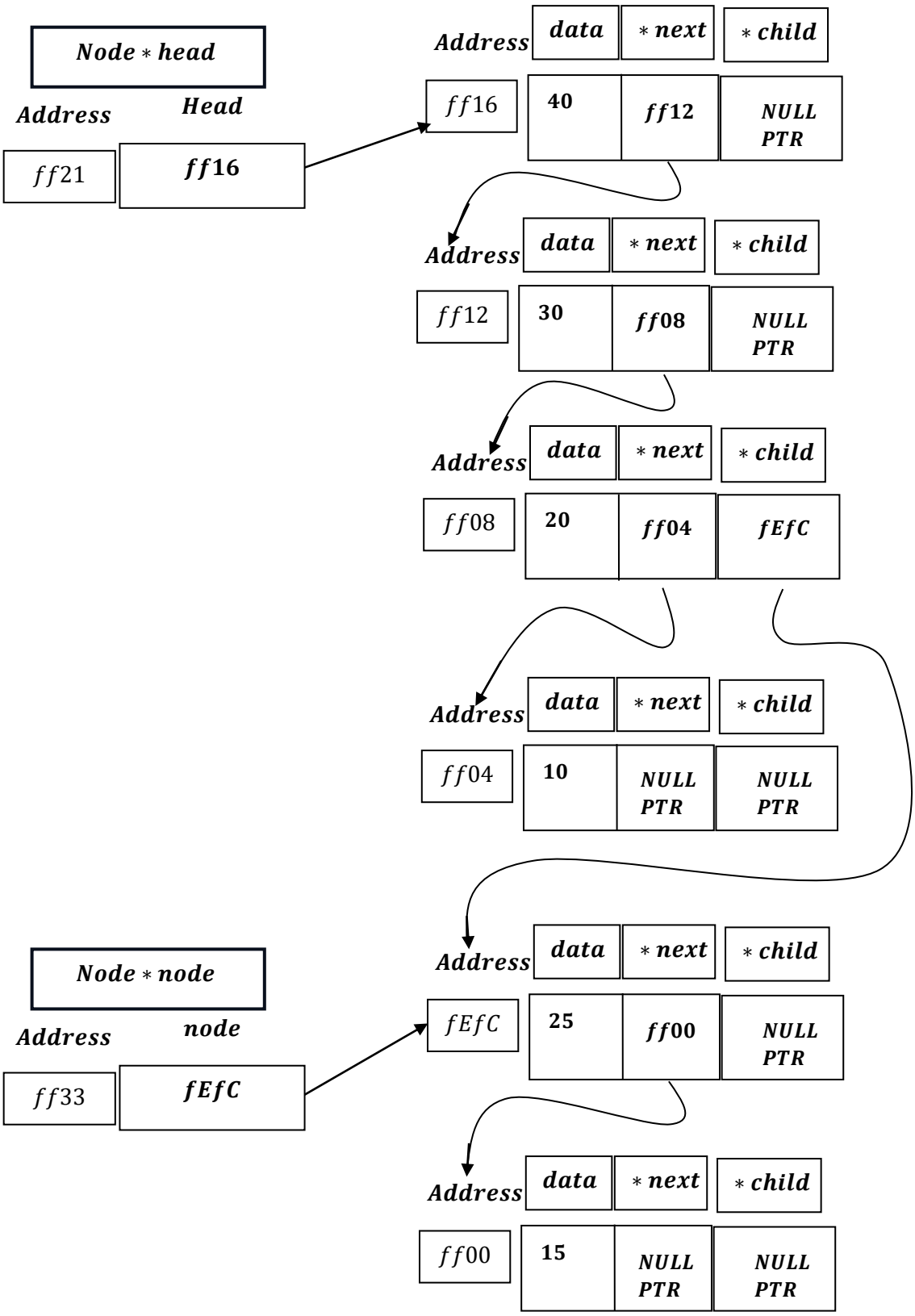
$$|--30$$

$$|--20$$

Assigning after recursive call:

$$\boldsymbol{void\ print(Node * node: fEfC, int\ Level = 1)}$$

$$\boldsymbol{printList(node \rightarrow child: fEfC, Level: 0 + 1 = 1)}$$



$$\textbf{\textit{Recursive Call Runs}}:$$

$$\textbf{\textit{while node}}: fEfC \neq \textbf{\textit{Nullptr}}\ ,\textbf{\textit{which is true}}:$$

$$for(\ int\ i = 0\ ; i: 0 < level: 1\ [i.e.\ i = 0]) \rightarrow$$

$$cout \ll ``\quad ``;$$

$$\textbf{\textit{It will create space as it already in the next line}}$$

$$\textbf{\textit{after last endl was executed.}}:$$

$$|--40$$

$$|--30$$

$$|--20$$

$$[Space]$$

$$\textbf{\textit{Now loop increment}}: i = i + 1 = 0 + 1 = 1$$

$$for(\ int\ i = 1\ ; i: 1 < level: 1\ [i.e.\ i = 0]) \begin{bmatrix} \textbf{\textit{Loop Condition}} \\ \textbf{\textit{becomesfalse}} \end{bmatrix}$$

$$[\textbf{\textit{Lower Bound}} > \textbf{\textit{Upper Bound}}: \textbf{\textit{Hence loop exit}}]$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node: fEfC \rightarrow data: 25 \ll endl;$$

$$|--40$$

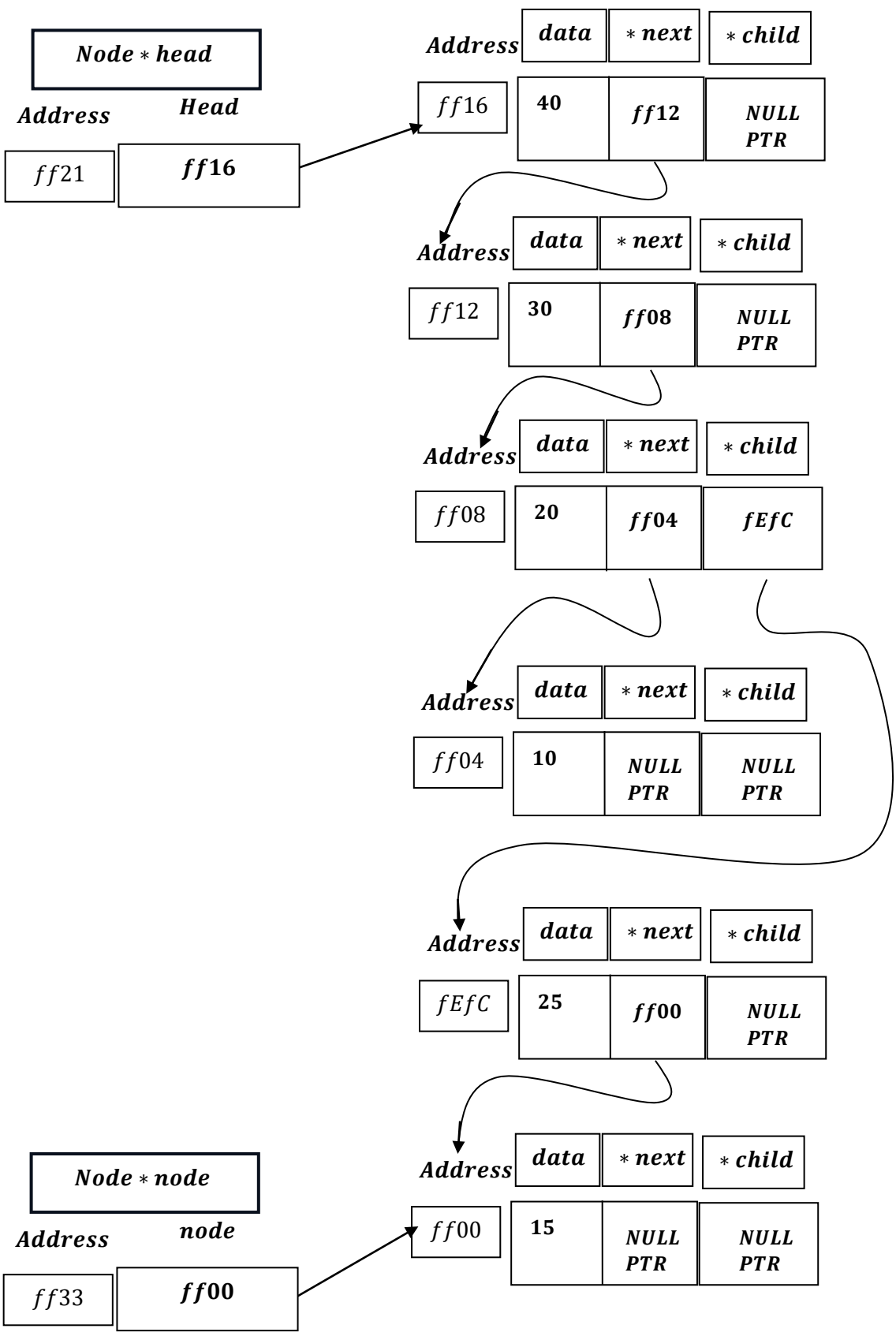
$$|--30$$

$$|--20$$

$$\boxed{[Space]} \quad |--25$$

$$if(node{:}\ fEfC \rightarrow child{:}\ NULLPTR \neq NULLPTR) which\ is\ false.$$

$$Node = Node{:}\ fEfC \rightarrow next{:}\ ff00$$


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR
Node * node
Address
node
ff33
ff00

$$while\ node{:}\ ff00\ \neq Nullptr\ , which\ is\ true{:}$$

$$for(\ int\ i = 0\ ;i{:}\ 0 < level{:}\ 1\ [i.e.i = 0]) \rightarrow$$

$$cout \ll ``\quad ``;$$

$$It\ will\ create\ space\ as\ it\ already\ in\ the\ next\ line$$
$$after\ last\ endl\ was\ executed.:$$

$$|--40$$
$$|--30$$
$$|--20$$
$$[Space]\quad |--25$$
$$[Space]$$

$$Now\ loop\ increment: i = i + 1 = 0 + 1 = 1$$

$$for(\ int\ i = 1\ ;i{:}\ 1 < level{:}\ 1\ [i.e.i = 0]) \begin{bmatrix} Loop\ Condition \\ becomes false \end{bmatrix}$$
$$[Lower\ Bound > Upper\ Bound{:}\ Hence\ loop\ exit]$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node: ff00 \rightarrow data: 15 \ll endl;$$

$$|--40$$

$$|--30$$

$$|--20$$

$$[Space] \quad |--25$$

$$[Space] \quad |--15$$

$$if(node: fEfC \rightarrow child: NULLPTR \neq NULLPTR) which\ is\ false.$$

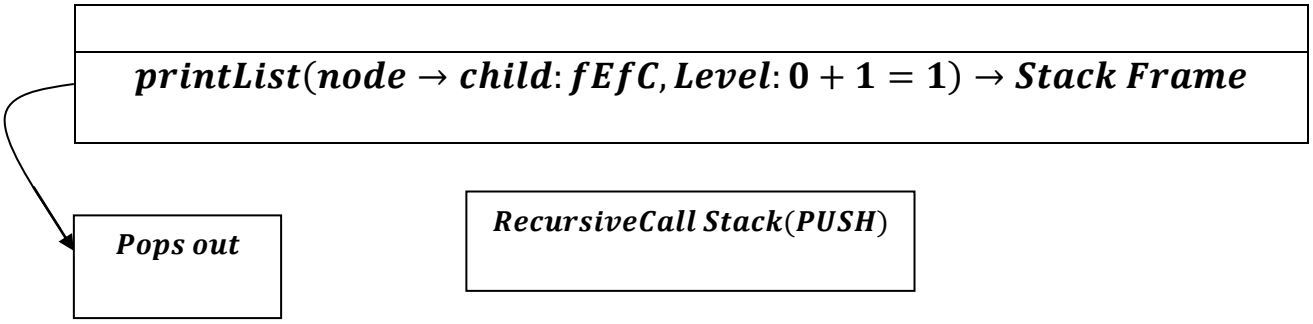
$$Node = Node: ff00 \rightarrow next: NULLPTR$$

$$while\ node: ff00 \neq Nullptr\ , which\ is\ true:$$

$$Now, as\ soon\ as\ , While\ loop's\ condition\ becomes\ NULLPTR:$$

$$Which\ means\ , Next\ and\ Child\ both\ becomes\ NULLPTR\ , now\ it\ reaches\ its\ base\ case.$$

$$As\ it\ reaches\ its\ base\ case, the\ recursive\ stack\ frame\ pops\ out:$$



Next it Back Tracks to :

$$printList(node: ff08 \rightarrow next: ff04, Level: 0)$$

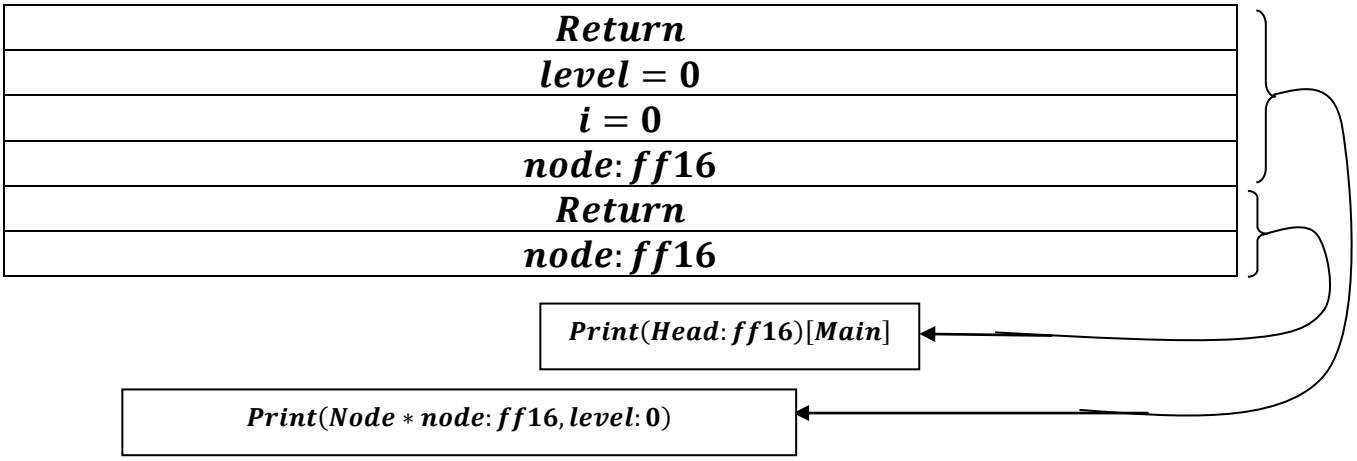
Now we have to understand, it is going under a loop of a single stack frame:

That is when the function call :

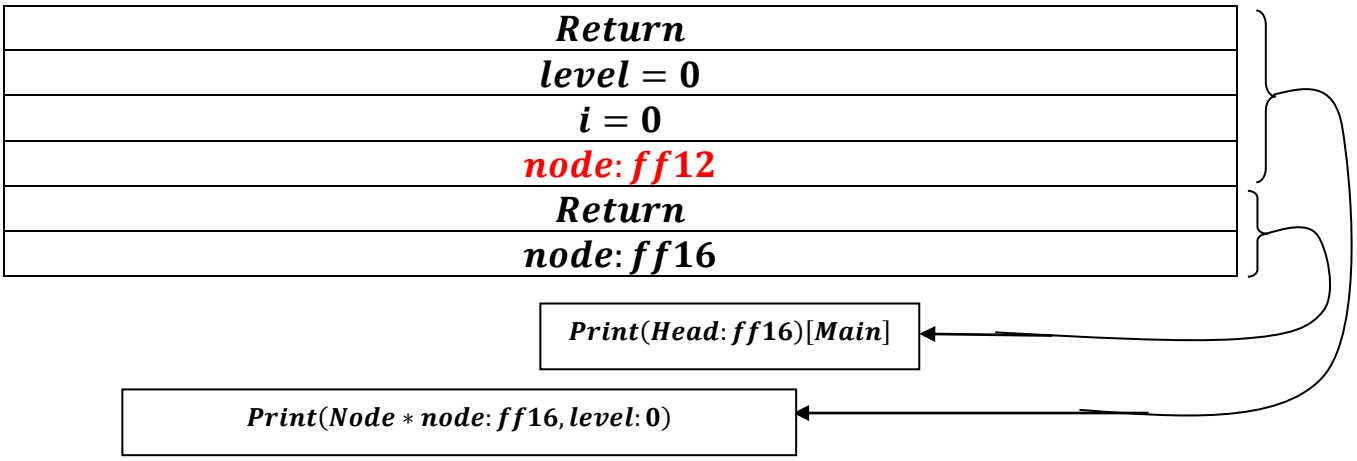
```
printList(head);
```

Return
$node: ff16$

$$Print(Head: ff16)[Main]$$

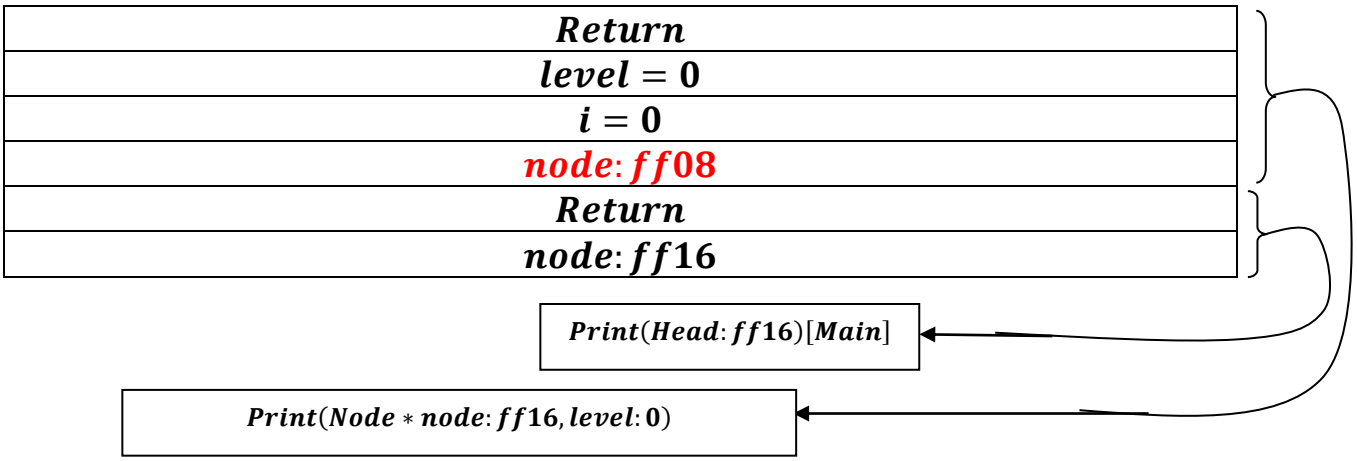


$Node: ff16 \rightarrow next: ff12$



Changes made in same stack frame , as it is while loop.

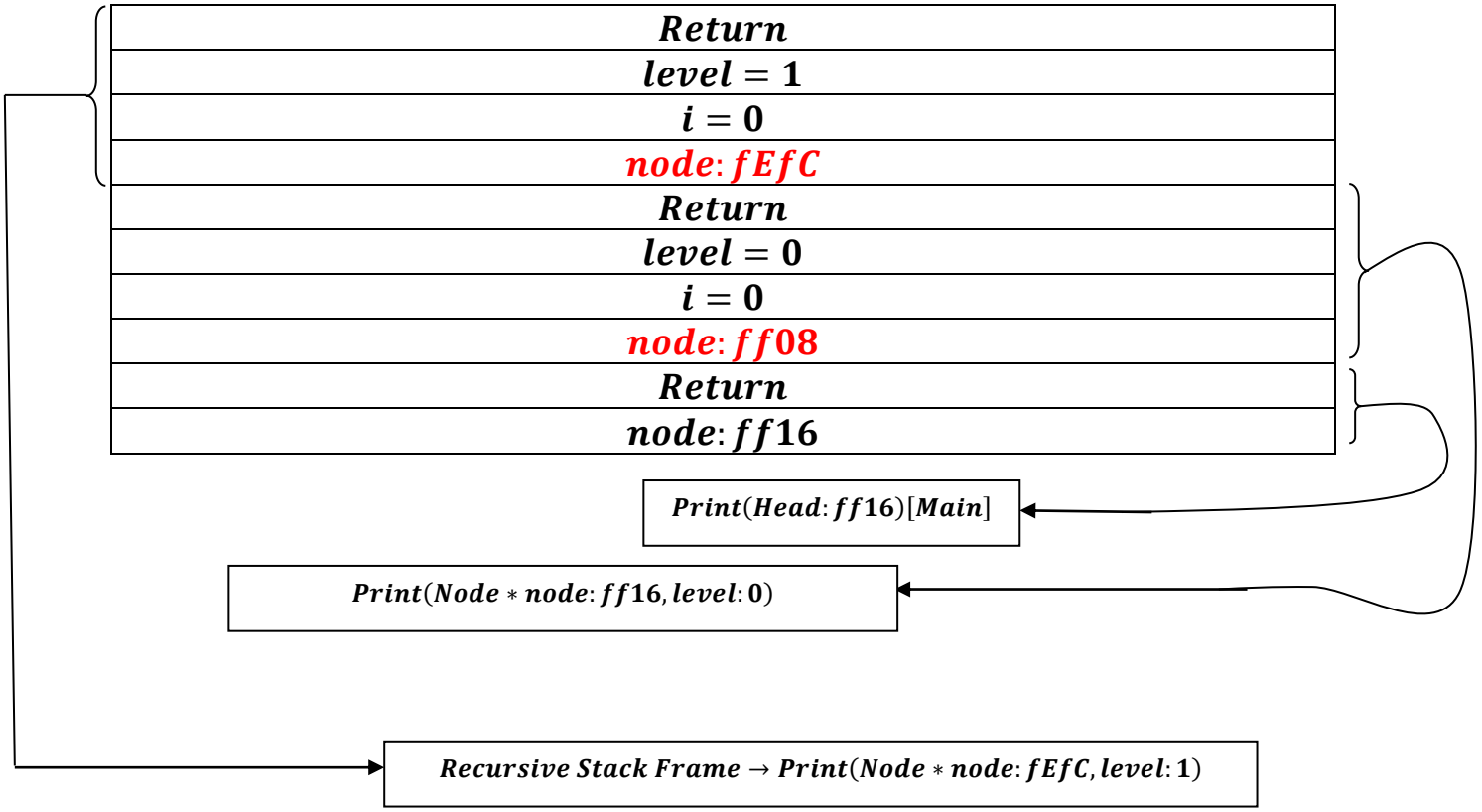
$$Node: ff12 \rightarrow next: ff08$$



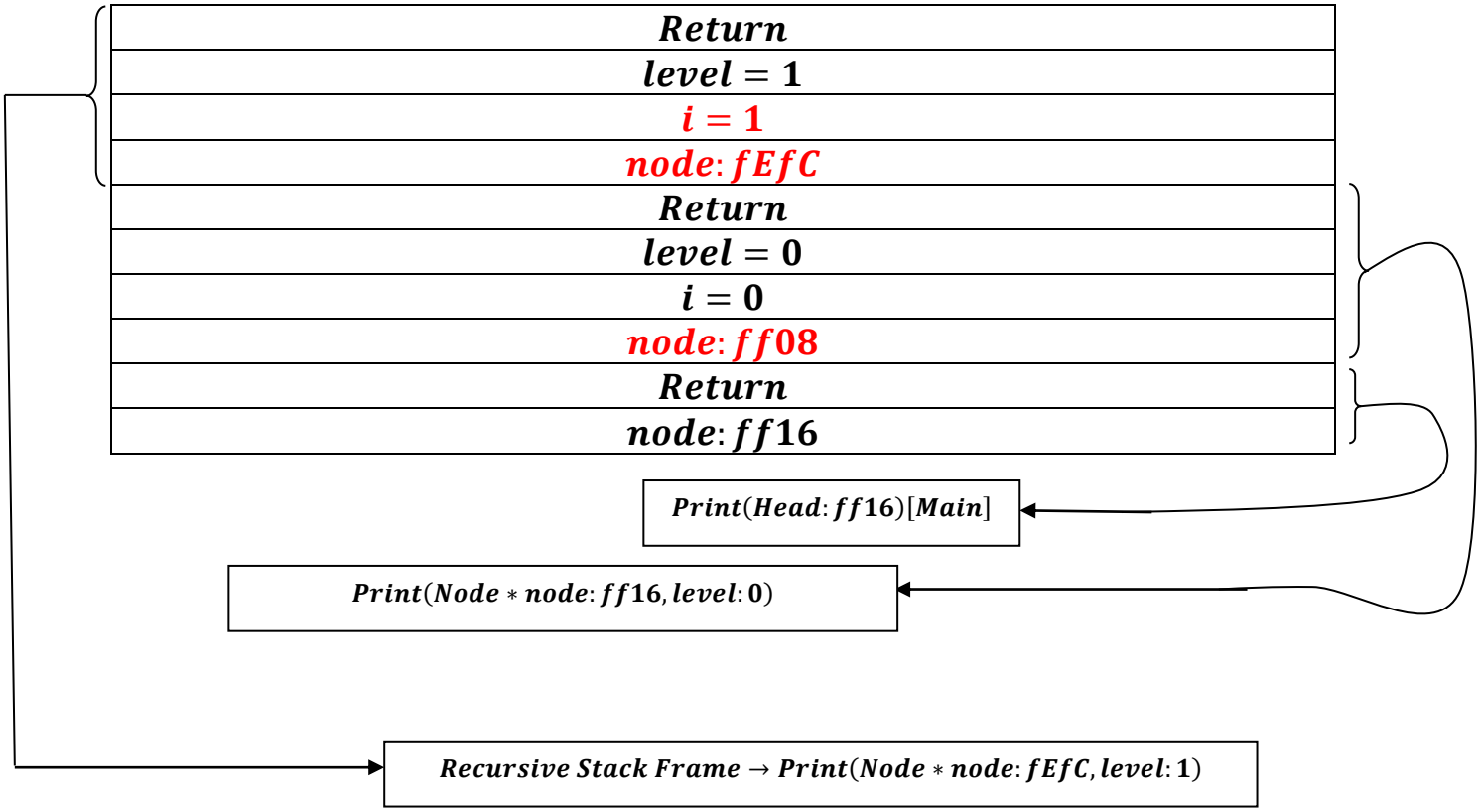
Changes made in same stack frame , as it is while loop.

Now as soon as , there is a Recursive call:

$$printList(node \rightarrow child: fEfC, Level: 0 + 1 = 1)$$



***As For loop runs*:**

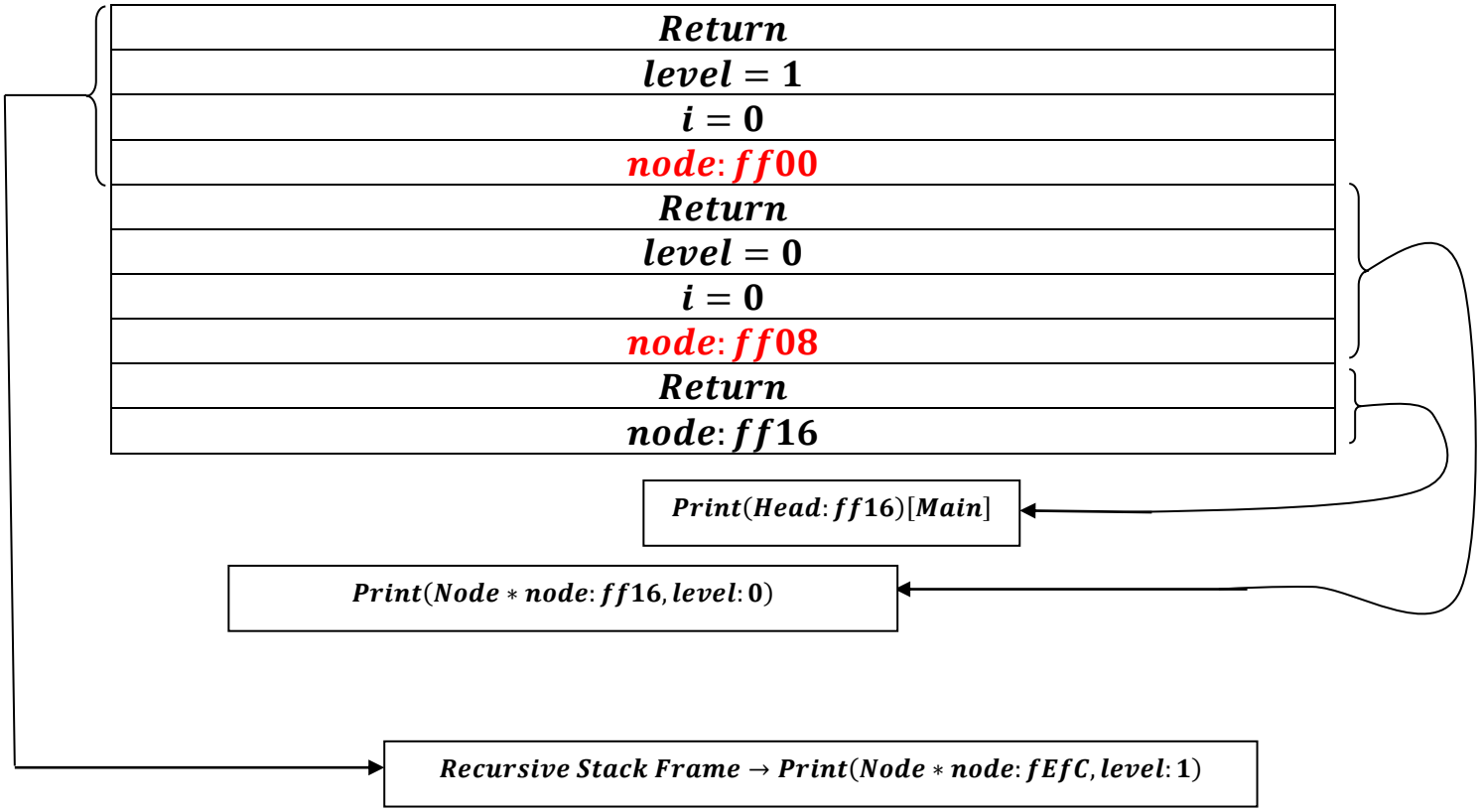


$Node: fEfC \rightarrow next: ff00$

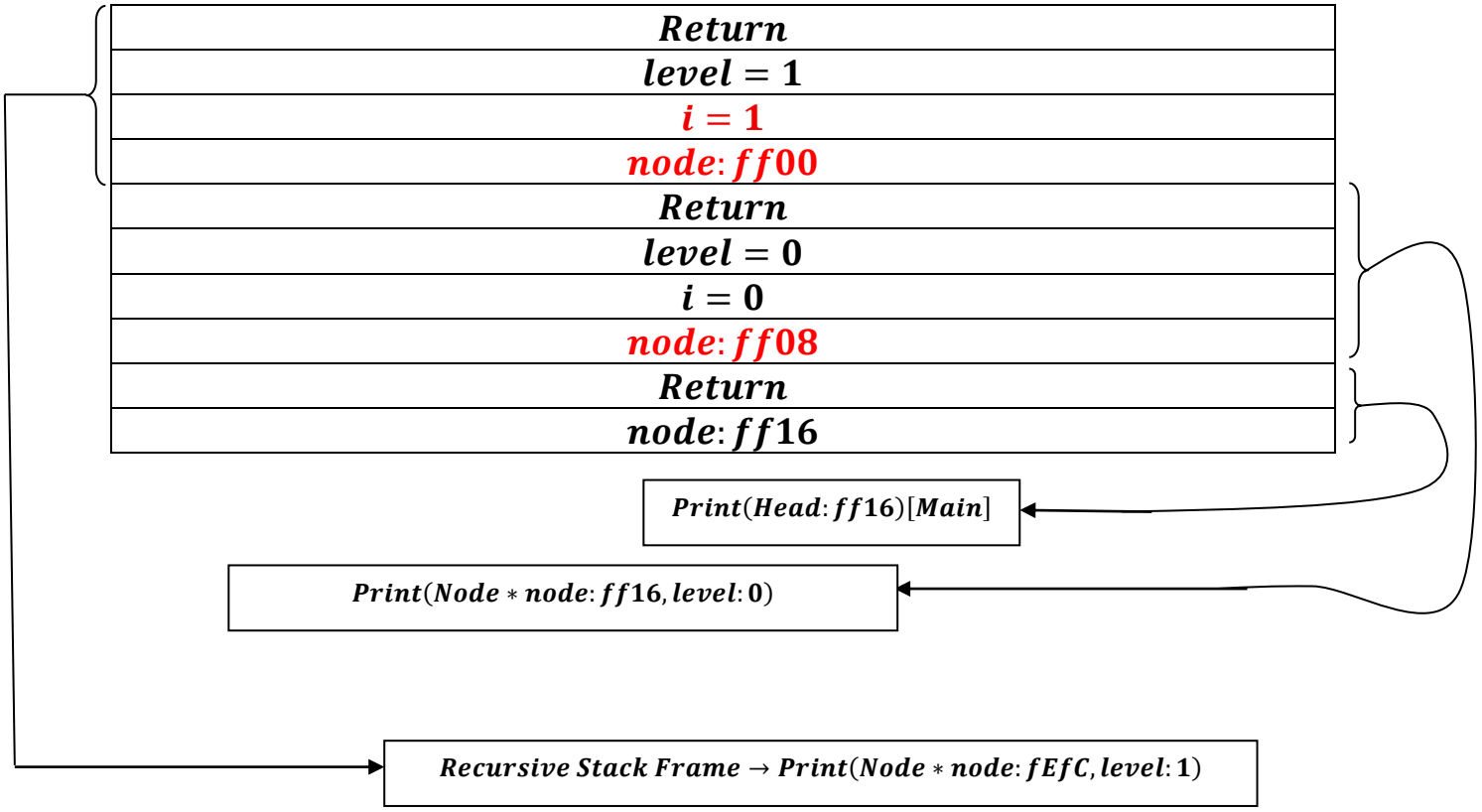
This change occur in same recursive stack frame:

$Print(Node * node: fEfC, level: 1)$

As, it is executed through while loop not another recursive call.



As , for loop gets executed :

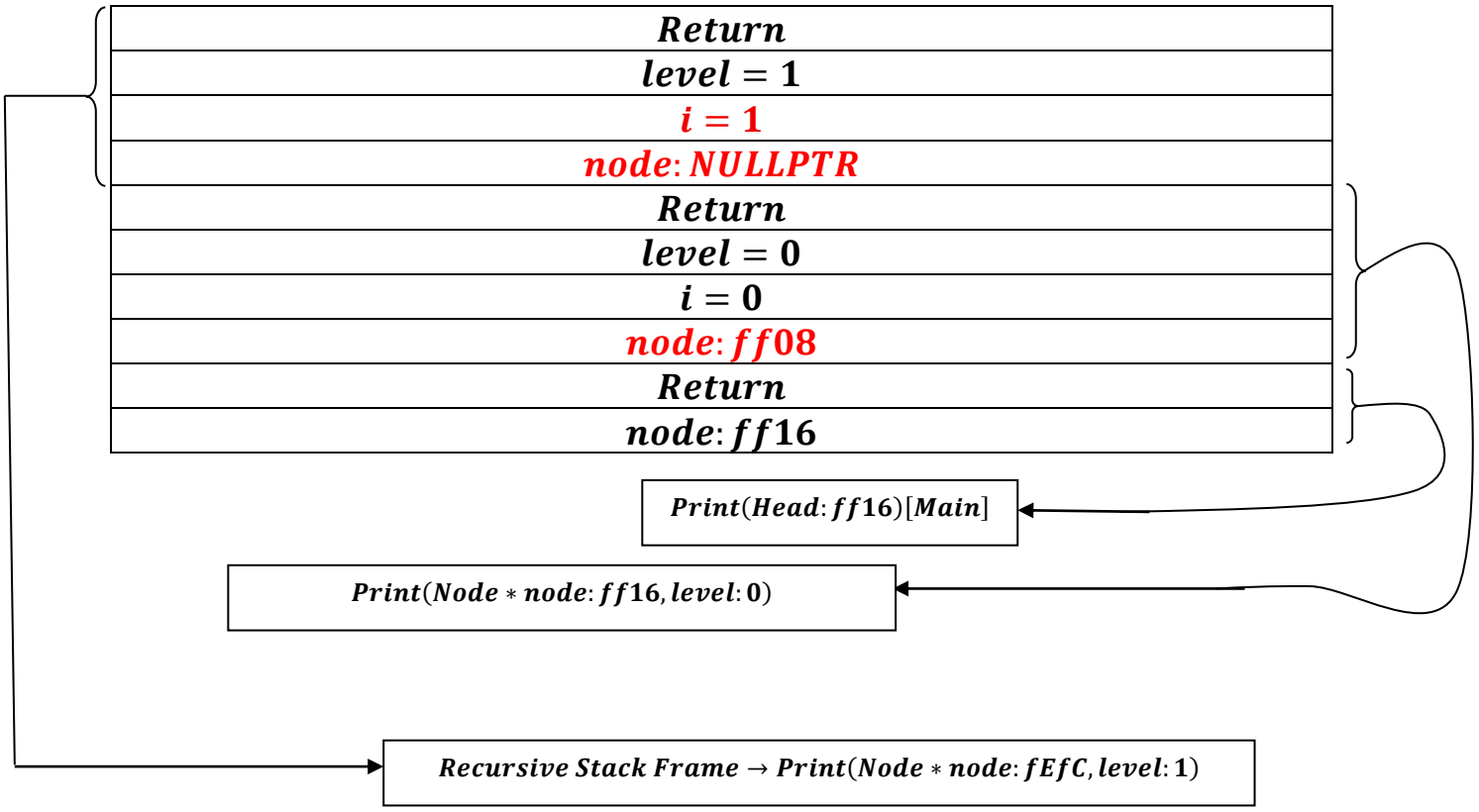


$$Node: ff00 \rightarrow next: NULLPTR$$

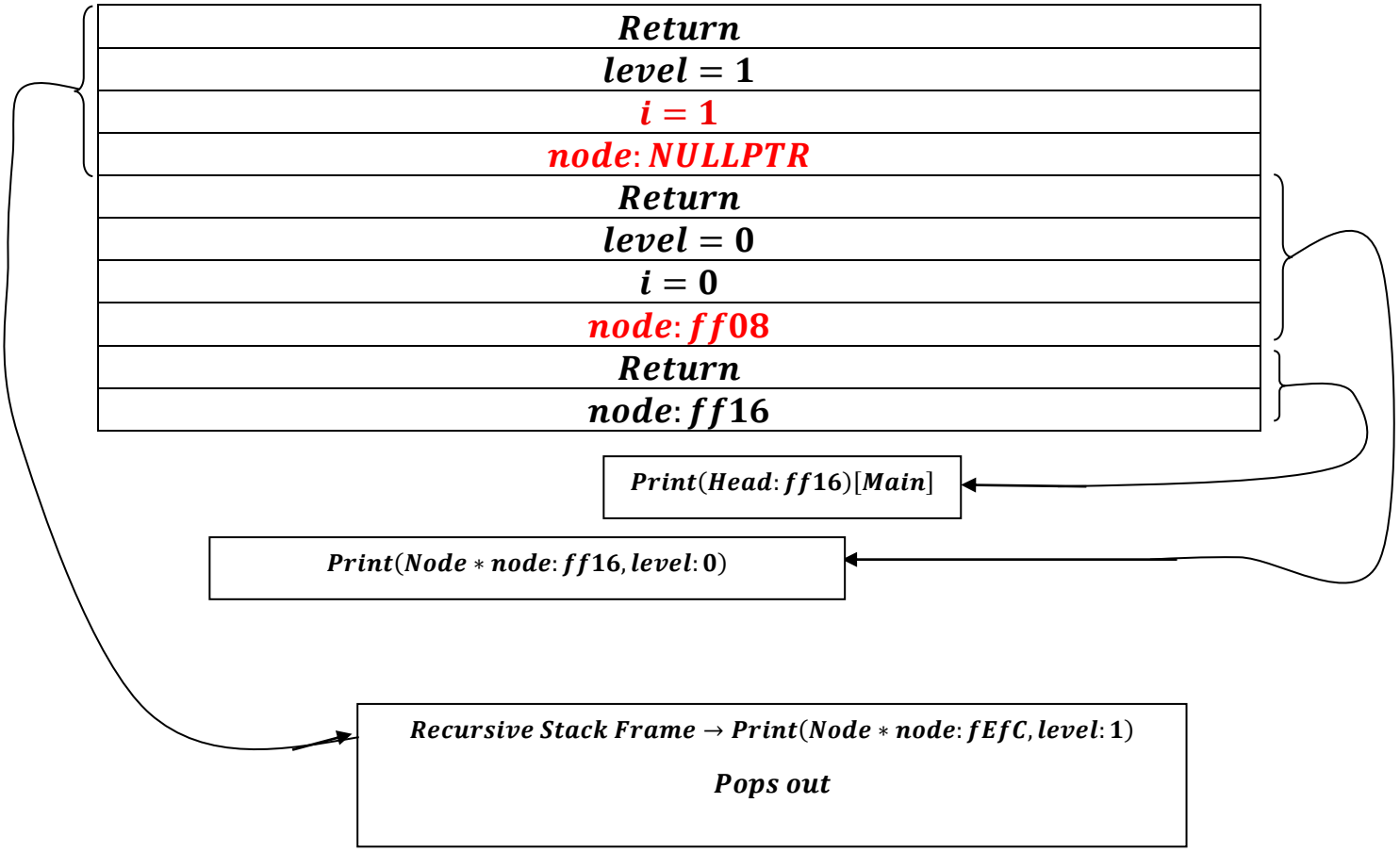
This change occur in same recursive stack frame:

$Print(Node * node: fEfC, level: 1)$

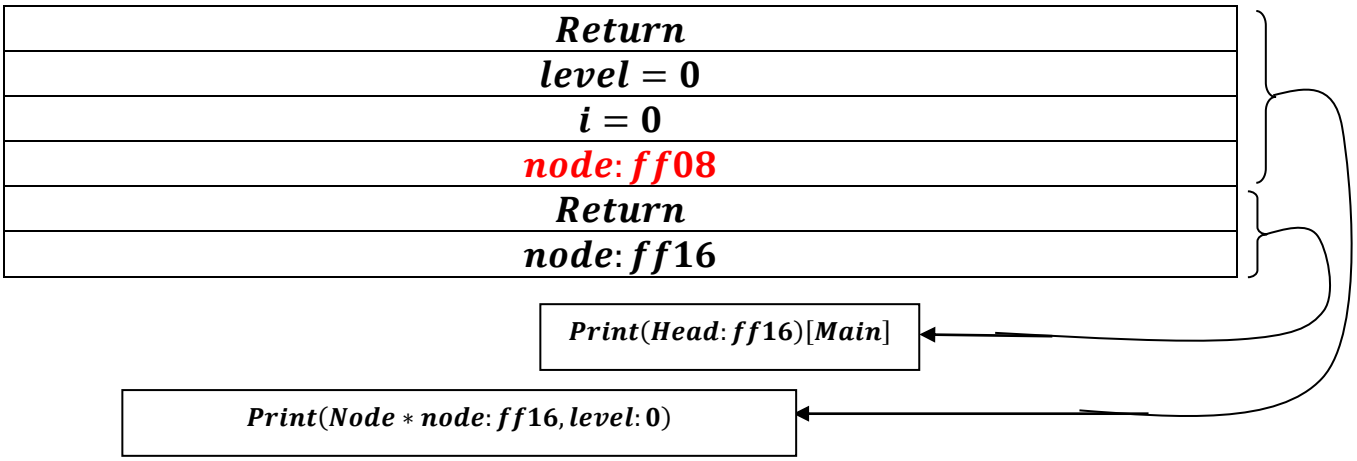
As, it is executed through while loop not another recursive call.



As soon as , NODE = NULLPTR and While Loop fails, , this recursive stack frame $Print(Node * node: fEfC, level: 1)$ ***produced from*** $Print(node \rightarrow child: fEfC, level + 1)$ ***gets destroyed or one can say popped out(due to*** $child \rightarrow nullptr$ ***and*** $next \rightarrow nullptr$ ***, hence base case reached***).



Then it backtracks to Print(Node * node: ff16, level : 0) stack frame, through the help of return , where Node * node : value update was stored ff08.



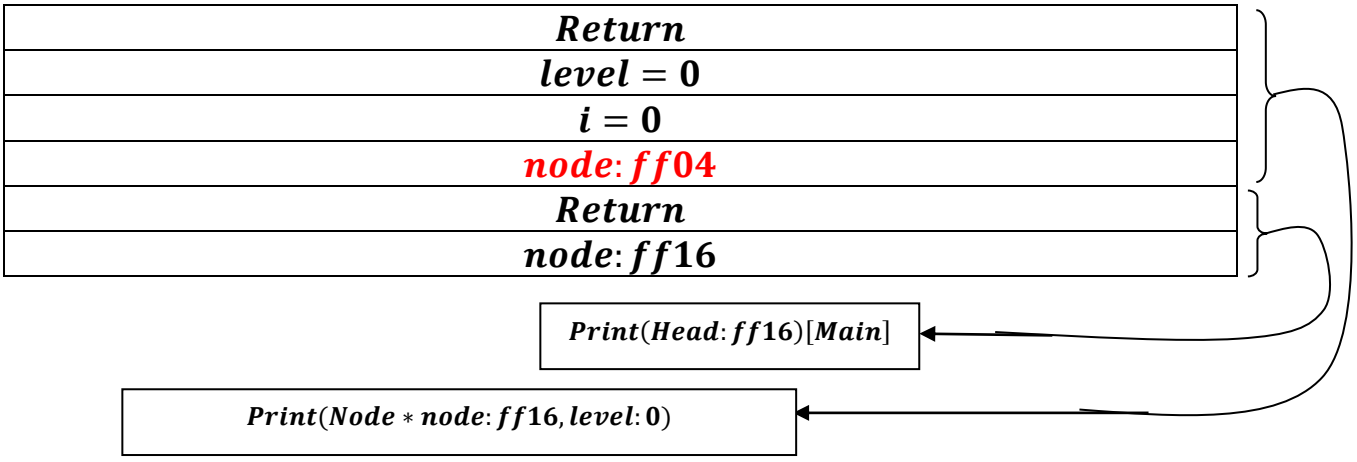
It will then run , next line of code:

```
        node = node->next;
```

$$node = Node: ff08 \rightarrow Next: ff04$$

from where it was paused earlier , while running recursion:

$$Print(node \rightarrow child: fEfC, level + 1)$$



$$while\ node{:}\ ff04\ \neq Nullptr\ , which\ is\ true{:}$$

$$for(\ int\ i = 0\ ; i{:}\ 0 < level{:}\ 0\ [false\ condition]; i++) \rightarrow$$

$$(condition\ is\ false, hence\ loop\ doesn't\ run)$$

```
        cout << "|-- " << node->data << endl;
```

$$cout \ll ``|--`` \ll node{:}\ ff04 \rightarrow data{:}\ 10 \ll endl;$$

$$if(node{:}\ ff04 \rightarrow child{:}\ Nullptr \neq NULLPTR) which\ is\ false\ .$$

$$Node = Node \rightarrow next{:}\ NULLPTR.$$

$$|--40$$

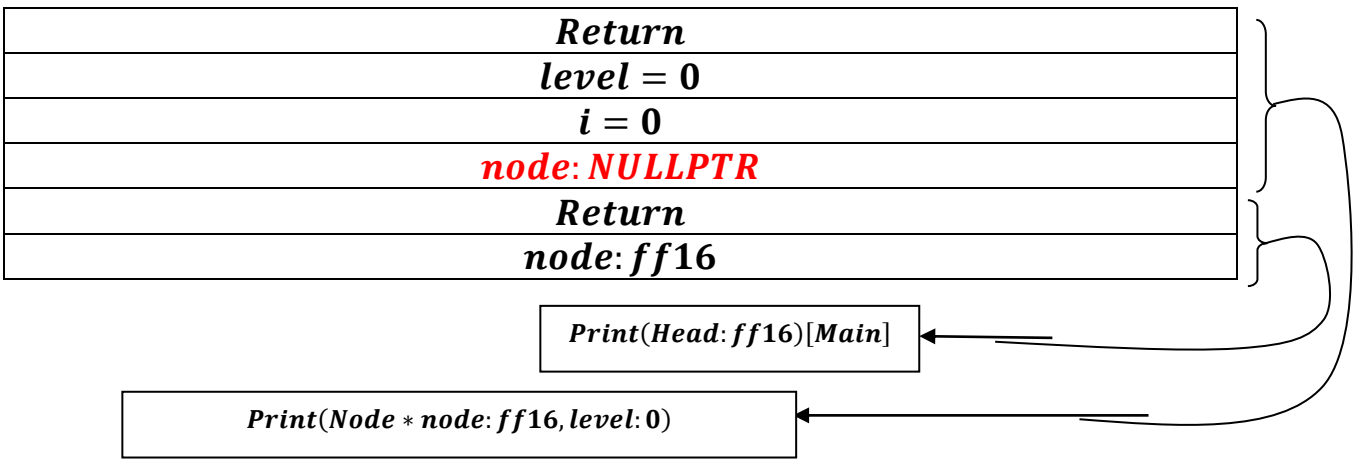
$$|--30$$

$$|--20$$

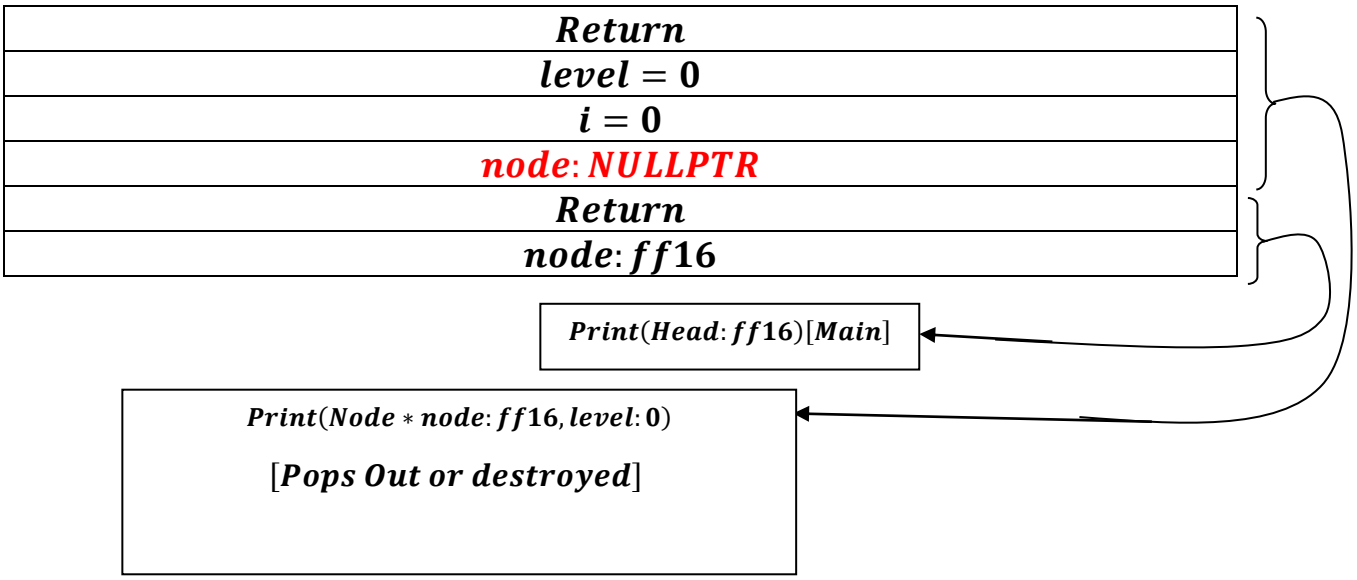
$$[Space] \quad |--25$$

$$[Space] \quad |--15$$

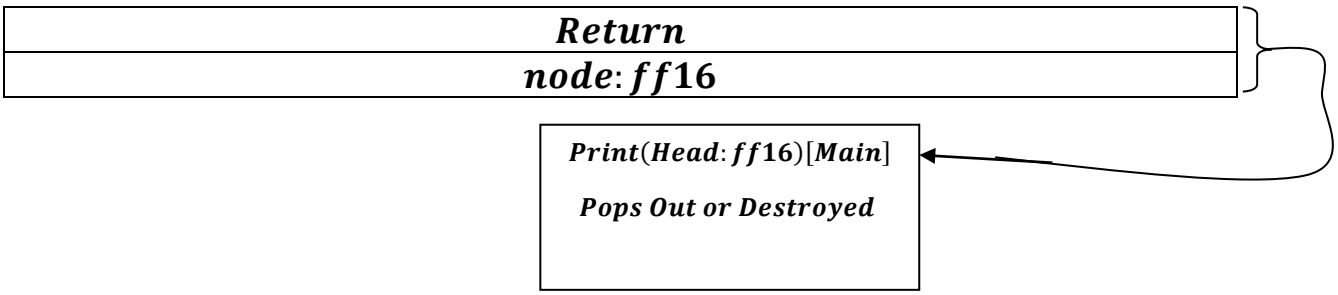
$$|--10$$



As soon as , NODE = NULLPTR and While Loop fails, , this recursive stack frame Print(Node * node: ff16, level: 0) gets destroyed or one can say popped out(due to child → nullptr and next → nullptr , hence base case reached).



Finally, as function gets finished:



Therefore ,we get finally our print list:

$$|--40$$

$$|--30$$

$$|--20$$

$$[Space] \quad |--25$$

$$[Space] \quad |--15$$

$$|--10$$
